11장 서브모듈

지금까지 우리는 저장소 하나로만 작업했습니다. 대부분의 프로젝트는 깃 저장소 하나만으로도 충분히 관리할 수 있습니다. 이 장에서는 저장소 하나를 여러 저장소로 분리하는 방법을 살펴보겠습니다.

11.1 대형 프로젝트

11.2 실습을 위한 저장소 준비

11.3 서브모듈 추가

11.4 서브모듈 작업

11.5 자식 저장소 갱신

11.6 부모 저장소 복제

11.7 부모 저장소 업데이트

11.8 정리

11.1 대형 프로젝트

깃은 작은 프로젝트부터 대용량 프로젝트까지 관리할 수 있는 버전 관리 도구입니다. 깃은 작은 프로젝트뿐만 아니라 큰 규모의 프로젝트도 관리하기 쉽도록 저장소를 여러 개 이용하여 단일 프로젝트를 진행할 수 있습니다.

11.1.1 저장 용량

깃 저장소는 용량 한계가 없습니다. 컴퓨터의 하드디스크에서 지원하는 용량만큼 깃 저장소를 생성할 수 있습니다. 또 깃은 파일을 기반으로 데이터베이스를 관리합니다. 하지만 저장소 크기가 너무 크면 컴퓨터에 부담을 줍니다.

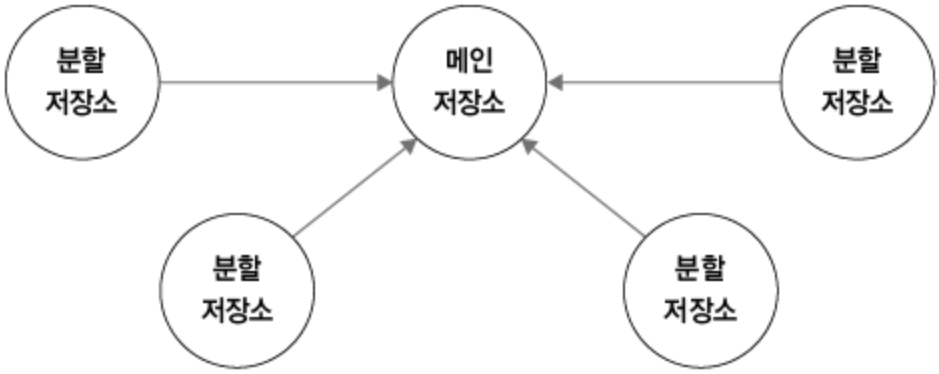
이러한 이유로 깃 호스팅 서비스들은 제공되는 저장소 용량을 제한합니다. 보통 1GB 용량을 제공하기 때문에 파일 개수가 많은 프로젝트에서는 좀 더 효율적인 관리 방법이 필요합니다.

11.1.2 저장소 분리

규모가 큰 프로젝트는 저장소 용량이 많이 필요합니다. 또 단지 용량이 크기만 한 저장소는 비효율적입니다. 필요에 맞게 작은 저장소로 분할하여 운영하는 것이 좀 더 효율적입니다. 깃 역시 큰 규모의 저장소를 작은 저장소로 나누어 관리할 수 있습니다.

즉, 저장소 하나를 작은 저장소 몇 개로 분할할 수 있다는 의미입니다. 이러한 저장소의 분할 개념을 서브모듈이라고 합니다. 서브모듈은 저장소 하나가 다른 깃 저장소를 포함하는 형태를 의미합니다. 이러한 서브모듈 개념을 적용하면 대형 프로젝트도 효과적으로 관리할 수 있습니다.

▼ 그림 11-1 분할된 저장소



작게 나뉜 모듈 형태의 저장소는 다른 프로젝트에서도 재사용할 수 있습니다.

11.1.3 상하 관계

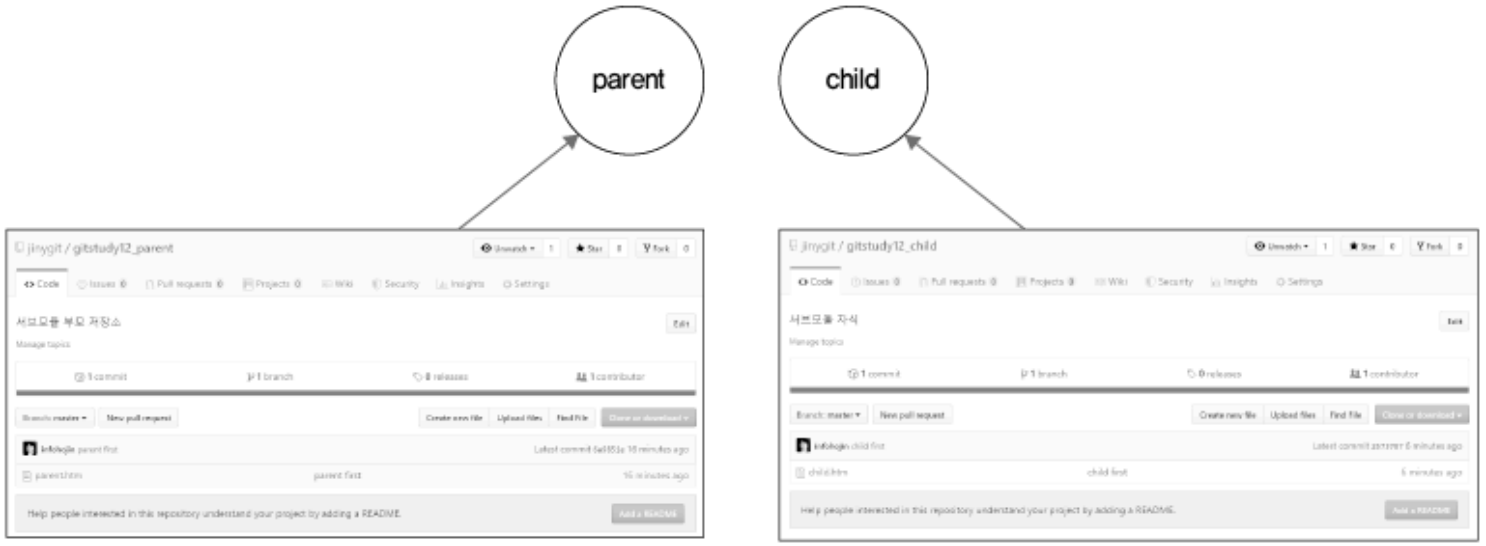
요즘 규모가 큰 프로젝트는 모듈화하여 개발하는 추세입니다. 각 기능들을 모듈화하여 독립된 깃 저장소로 관리합니다. 그리고 독립된 깃 저장소는 모듈로서 다시 메인 저장소와 결합하여 재사용됩니다.

메인 저장소에는 서브 저장소가 여러 개 있습니다. 따라서 저장소 간 상하 관계가 발생합니다. 보통 부모 저장소와 자식 저장소 형태로 나눕니다. 부모 자식 관계로 분리된 로컬 저장소는 또다시 개별 원격 저장소와 연결됩니다. 원격 저장소로 동기화된 자식들은 언제든지 다른 프로젝트의 모듈별로 기능을 처리하고 결합할 수 있습니다.

11.2 실습을 위한 저장소 준비

서브모듈을 실습할 수 있는 환경을 구축하겠습니다. 서브모듈을 테스트하려면 저장소가 2개 필요합니다. 먼저 로컬 컴퓨터에 깃 저장소를 2개 생성할 것입니다. 그리고 원격 저장소도 2개 생성하여 각 로컬 저장소와 연결하겠습니다.

▼ 그림 11-2 깃 저장소 생성 후 원격 저장소와 연결



11.2.1 메인 저장소 생성

먼저 부모 역할의 메인 저장소를 생성합니다.

```
$ cd 실습폴더


$ mkdir gitstudy11_parent 새 폴더 만들기


$ cd gitstudy11_parent
```

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)


$ git init 저장소 초기화


Initialized empty Git repository in E:/gitstudy11_parent/.git/
```

메인 저장소(gitstudy11_parent)에 parent.htm 파일을 생성하고 저장합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)


$ code parent.htm VS Code 실행
```

parent.htm

```
<h1>부모 저장소</h1>
```

첫 번째 커밋을 합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)

$ git add parent.htm 등록




infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)

$ git commit -m "parent first" 커밋

[main (root-commit) 6a9851e] parent first

 1 file changed, 1 insertion(+)

 create mode 100644 parent.htm
```

11.2.2 자식 저장소 생성

이번에는 메인 저장소와 연결되는 자식 저장소를 생성합니다.

```
$ cd 실습폴더


$ mkdir gitstudy11_child 새 폴더 만들기


$ cd gitstudy11_child
```

```
infoh@DESKTOP MINGW64 /e/gitstudy11_child (main)


$ git init 저장소 초기화


Initialized empty Git repository in E:/gitstudy11_child/.git/
```

자식 저장소에 child.htm 파일을 생성하고 저장합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_child (main)


$ code child.htm VS Code 실행
```

child.htm

```
<h1>자식 저장소</h1>
```

자식 저장소에서 첫 번째 커밋을 합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_child (main)

$ git add child.htm 등록




infoh@DESKTOP MINGW64 /e/gitstudy11_child (main)

$ git commit -m "child first" 커밋

[main (root-commit) 2573787] child first

 1 file changed, 1 insertion(+)

 create mode 100644 child.htm
```

11.3 서브모듈 추가

두 저장소를 생성하고 원격 저장소와 연결하여 서브모듈 실습 환경을 준비했습니다. 서브모듈은
2개 이상인 저장소를 부모와 자식 관계로 연결합니다.

저장소를 모듈로 연결하려면 submodule 명령어를 사용합니다.

$ git submodule -help

submodule 명령어는 옵션을 사용하여 다양한 동작을 실행할 수 있습니다. 자세한 옵션은 -help 명령어로 확인합니다.

11.3.1 저장소 연결

메인 저장소에 자식 저장소를 연결합니다. 메인 저장소에 자식 저장소를 추가하는 옵션은 add 명령어입니다. 서브모듈로 저장소가 추가되면 메인 저장소는 서브 저장소를 서브 폴더 형태로 취급합니다.

$ git submodule add 원격저장소URL 폴더이름

실습하면서 개념을 익혀 봅시다. 먼저 서브 저장소가 추가될 메인 저장소로 이동합니다. 여기서는 부모 저장소를 의미합니다.



```
infoh@DESKTOP MINGW64 /e/gitstudy11_child (main)


$ cd ../gitstudy11_parent 메인 저장소로 이동




infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)
```

메인 저장소에 submodule 명령어를 사용하여 또 다른 저장소를 추가합니다. 실습에서는 준비한 자식 저장소를 등록하겠습니다. 자식 저장소를 등록할 때는 자식 저장소와 연결된 원격 저장소 주소를 입력합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)

$ git submodule add https://github.com/jinygit/gitstudy11_child.git child 자식 저장소 등록

Cloning into 'E:/gitstudy11_parent/gitstudy11_child'... 저장소 복제

remote: Enumerating objects: 3, done.

remote: Counting objects: 100% (3/3), done.

remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0

Unpacking objects: 100% (3/3), done.

warning: LF will be replaced by CRLF in .gitmodules.

The file will have its original line endings in your working directory
```

11.3.2 설정 파일

처음으로 메인 저장소에 서브모듈이 등록되면 깃은 루트 위치에 설정 파일을 생성합니다. 설정 파일은 메인 저장소와 연결된 자식 저장소들을 관리합니다. 설정 파일 이름은 .gitmodules입니다. 파일 이름이 점(.)으로 시작되기 때문에 숨김 파일 형태로 관리합니다.

설정 파일 내용을 살펴봅시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)


$ cat .gitmodules 서브모듈 설정 파일


[submodule "child"]


        path = child 폴더 이름


        url = https://github.com/jinygit/gitstudy11_child.git 원격 주소 URL
```

환경 설정에는 서브모듈 이름과 URL이 기록되어 있군요.

11.3.3 모듈 커밋

서브모듈 명령을 실행하여 메인 저장소에 자식 저장소를 연결했습니다. 서브모듈이 추가된 부모 저장소의 상태를 확인해 보겠습니다. status 명령어를 입력합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)


$ git status 상태 확인


On branch main


Changes to be committed:


  (use "git reset HEAD <file>..." to unstage)


        new file:   .gitmodules


        new file:   child
```

앞에서 확인했듯이, 메인 저장소에 자식 저장소의 폴더와 환경 설정 파일을 생성했습니다. 메인 저장소와 서브모듈인 자식 저장소 간 관계를 지속적으로 유지하려면 추가된 정보들을 계속 가지고 있어야 합니다. 메인 저장소가 자식들의 정보를 계속 가지려면 이를 커밋하여 저장해야 합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)

$ git add .gitmodules 등록




infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)

$ git commit -m "add submodule" 커밋

[main 7a6d131] add submodule

 2 files changed, 4 insertions(+)

 create mode 100644 .gitmodules

 create mode 160000 child
```

11.4 서브모듈 작업

메인 저장소를 서브모듈 형태로 변경했습니다. 메인 저장소에는 서브 저장소의 연결 정보만 있습니다. 메인 저장소에 등록된 서브 저장소는 독립된 별도의 저장 공간입니다.

11.4.1 모듈 저장소

독립된 자식 저장소에서 새롭게 작업해 봅시다. 먼저 메인 저장소 내용을 확인해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)
```

```
$ ls 파일 목록
```

```
child/ parent.htm
```

메인 저장소에 등록된 서브모듈의 폴더로 이동합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)
```

```
$ cd child 서브모듈 폴더
```

child는 자식 저장소의 원격 저장소를 복제한 폴더입니다. 서브 폴더 안의 내용을 확인합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent/child (main)
```

```
$ ls -all 서브모듈의 파일 목록
```

```
total 6
```

```
drwxr-xr-x 1 infoh 197609  0 5월  25 15:27 .
```

```
drwxr-xr-x 1 infoh 197609  0 5월  25 15:27 ..
```

```
-rw-r--r-- 1 infoh 197609 30 5월  25 15:27 .git
```

```
-rw-r--r-- 1 infoh 197609 25 5월  25 15:27 child.htm
```

child 폴더 안에 또 다른 숨긴 저장소인 .git 폴더가 있습니다. 서브 폴더는 서브모듈로 분리한 독립된 깃 저장소입니다.

11.4.2 모듈 상태

모듈로 등록된 자식 저장소를 수정합시다. 메인 저장소의 child 폴더에서 child.htm 파일의 내용을 수정합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent/child (main)


$ code child.htm VS Code 실행
```

서브모듈 폴더

child.htm

```
<h1>자식 저장소</h1>


<h2>child는 서브모듈입니다.</h2>
```

파일을 수정한 후 서브 폴더 안에서 깃 상태를 확인합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent/child (main)

$ git status 서브 폴더 안에서 상태 확인

On branch main

Your branch is up to date with 'origin/main'.



Changes not staged for commit:

  (use "git add <file>..." to update what will be committed)

  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   child.htm

no changes added to commit (use "git add" and/or "git commit -a")
```

child 폴더 안의 파일을 수정했습니다. 서브 폴더도 독립된 깃 저장소이기 때문에 status 명령어로 상태를 추적할 수 있습니다.

11.4.3 모듈 커밋

parent/child 저장소의 내용을 수정했습니다. 자식의 서브 폴더 저장소에서 수정한 내용을 커밋합시다. 먼저 메인 저장소의 parent/child 폴더로 이동합니다.



```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)

$ cd child 서브모듈 폴더 이동

infoh@DESKTOP MINGW64 /e/gitstudy11_parent/child (main)

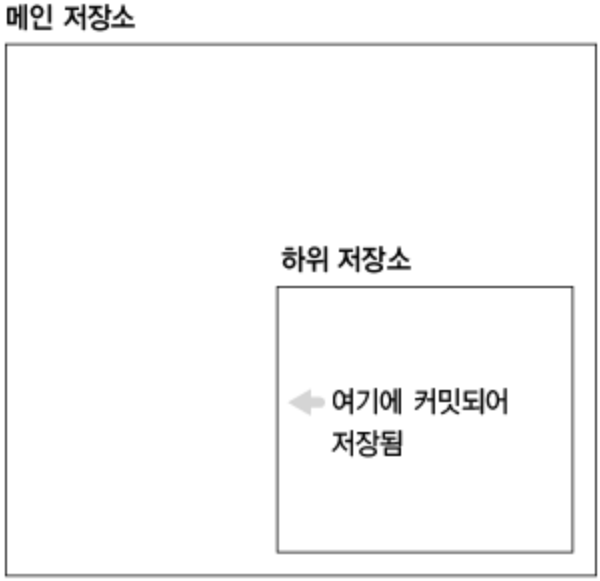
$ git commit -am "add content of child" 등록 및 커밋

[main a7709f5] add content of child

 1 file changed, 2 insertions(+), 1 deletion(-)
```

커밋했습니다. 이 커밋은 메인 저장소에서 하위로 복제 생성된 자식 저장소에만 커밋하는 것입니다. 저장소 상태를 다시 확인합니다.

▼ 그림 11-9 커밋해서 저장한 저장소 위치




```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent/child (main)

$ git status 서브 폴더의 상태 확인

On branch main

Your branch is ahead of 'origin/main' by 1 commit.

  (use "git push" to publish your local commits)



nothing to commit, working tree clean
```

11.4.4 부모 커밋

parent/child 폴더의 저장소에서 수정 작업을 했습니다. 이제 상위 메인 저장소로 이동하여 부모 저장소의 상태를 다시 확인해 보겠습니다.



```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent/child (main)

$ cd .. 메인 저장소로 이동



infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)

$ git status 상위 폴더의 상태 확인

On branch main

Your branch is ahead of 'origin/main' by 1 commit.

  (use "git push" to publish your local commits)



Changes not staged for commit:

  (use "git add <file>..." to update what will be committed)

  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   child (new commits)
```

```
no changes added to commit (use "git add" and/or "git commit -a")
```

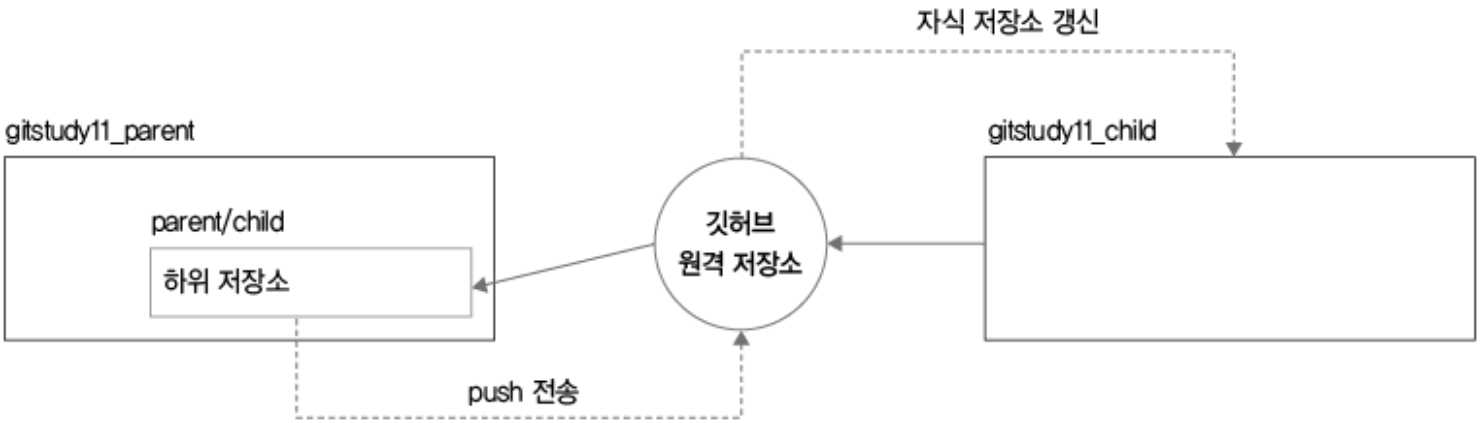
11.5 자식 저장소 갱신

지금까지 한 작업은 메인 저장소에 복제된 parent/child 저장소에서만 작업했습니다. 수정된 parent/child 저장소의 커밋들은 원격 저장소와 동기화했습니다.

11.5.1 자식 저장소

자식 저장소(gitstudy11_child)와 메인 저장소(parent/child)에 등록된 저장소는 서로 다른 영역입니다. gitstudy11_child 저장소는 처음에 생성한 자식 저장소고, 메인 저장소에 등록된 parent/child 원격 저장소로 동기된 복제본입니다.

▼ 그림 11-11 자식 저장소의 관계도



따라서 서로 다른 두 저장소는 아직 동일한 커밋 정보를 가지고 있지 않습니다. 자식 저장소에 연결된 원격 저장소를 이용하여 실제 자식 저장소(gitstudy11_child)를 최신 커밋 정보로 갱신해야 합니다.

11.5.2 자식 저장소 갱신

메인 저장소에 등록된 서브모듈(parent/child)의 커밋들은 자식의 원격 저장소로 푸시되었습니다. 이제는 수정 반영된 커밋들을 실제의 자식 저장소(gitstudy11_child)로 가져와야 합니다.

실제 자식 저장소로 이동합니다.



```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)


$ cd ../gitstudy11_child 자식 원본 저장소




infoh@DESKTOP MINGW64 /e/gitstudy11_child (main)
```

실제 자식 저장소에서 child.htm 파일 내용을 확인합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_child (main)


$ cat child.htm


<h1>자식 저장소</h1>
```

부모 저장소의 parent/child에서 수정된 내용을 실제 자식 저장소에는 반영하지 않았습니다. 자식 저장소의 원격 저장소를 이용하여 갱신합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_child (main)

$ git pull origin main 커밋 내려받기

From https://github.com/jinygit/gitstudy11_child

 * branch            main     → FETCH_HEAD

Updating 2573787..a7709f5

Fast-forward

 child.htm | 3 ++-

 1 file changed, 2 insertions(+), 1 deletion(-)
```

11.5.3 자식 저장소 작업

지금까지는 메인 저장소에서 복제한 자식 저장소(parent/child)에서 수정 작업을 했습니다. 그리고 수정 내역의 커밋을 자식 저장소의 원격 저장소를 거쳐 실제 자식 저장소에 반영했습니다. 이번에는 반대로 실제 자식 저장소에서 코드를 수정하고, 이를 자식 저장소의 원격 저장소를 거쳐 부모 저장소의 서브모듈(parent/child)로 적용하는 방법을 알아보겠습니다.

먼저 실제 자식 저장소에서 child.htm 파일을 수정하고 커밋합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_child (main)


$ code child.htm VS Code 실행
```

child.htm

```
<h1>자식 저장소</h1>


<h2>child는 서브모듈입니다.</h2>


<h2>엄마 100원 주세요.</h2>
```

```
infoh@DESKTOP MINGW64 /e/gitstudy11_child (main)


$ git commit -am "give me 100 wan" 등록 및 커밋


[main 8ecc866] give me 100 wan


 1 file changed, 2 insertions(+), 1 deletion(-)
```

11.5.4 부모 저장소 적용

실제 자식 저장소에서 변경한 커밋을 메인 저장소의 서브모듈(parent/child)에 반영해 보겠습니다. 로컬 저장소의 부모 저장소로 이동합니다.

메인 저장소로 이동합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_child (main)


$ cd ../gitstudy11_parent 부모 원본 저장소
```

그리고 메인 저장소에서 서브모듈의 자식 폴더로 이동합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)


$ cd child/ 서브모듈 폴더 이동
```

서브모듈 저장소의 코드 내용을 확인합시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent/child (main)


$ cat child.htm 확인


<h1>자식 저장소</h1>


<h2>child는 서브모듈입니다.</h2>
```

파일 내용은 이전 작업 상태 그대로입니다. 자식 저장소의 원격 저장소에서 최신 정보를 갱신합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent/child (main)

$ git pull origin main 커밋 내려받기

From https://github.com/jinygit/gitstudy11_child

 * branch            main     → FETCH_HEAD

Updating a7709f5..8ecc866

Fast-forward

 child.htm | 3 ++-

 1 file changed, 2 insertions(+), 1 deletion(-)
```

11.5.5 부모 저장소 갱신

자식 저장소의 원격 저장소를 이용하여 메인 저장소의 서브모듈을 갱신했습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent/child (main)


$ git status 상태 확인


On branch main


Your branch is up to date with 'origin/main'.




nothing to commit, working tree clean
```

서브모듈을 갱신하면 메인 저장소는 자신의 서브 폴더가 변경된 것을 인식합니다.

부모 폴더로 이동해서 상태를 확인해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent/child (main)

$ cd .. 메인 저장소 이동



infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)

$ git status 상태 확인

On branch main

Your branch is up to date with 'origin/main'.



Changes not staged for commit:

  (use "git add <file>..." to update what will be committed)

  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   child (new commits)

no changes added to commit (use "git add" and/or "git commit -a")
```

즉, 메인 저장소는 서브모듈의 변경 내용을 모니터링하고 있는 것입니다. 메인 저장소의 서브 폴더 자체를 변경했기 때문에 변경된 내용을 다시 메인 저장소로 커밋해야 합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11_parent (main)

$ git commit -am "update submodule" 등록 및 커밋

[main 6ebe93e] update submodule

 1 file changed, 1 insertion(+), 1 deletion(-)
```

11.6 부모 저장소 복제

서브모듈로 구성된 저장소는 부모/자식 관계를 맺습니다. 서브모듈의 저장소를 복제하거나 배포할 때는 부모/자식 관계도 같이 복제해야 합니다.

11.6.1 부모 저장소 복제

새로운 실습을 위해 clone 명령어를 실행하여 메인 저장소를 복제합니다.

```
$ cd 실습폴더


$ git clone https://github.com/jinygit/gitstudy11_parent.git gitstudy11 복제


Cloning into 'gitstudy11'...


remote: Enumerating objects: 8, done.


remote: Counting objects: 100% (8/8), done.

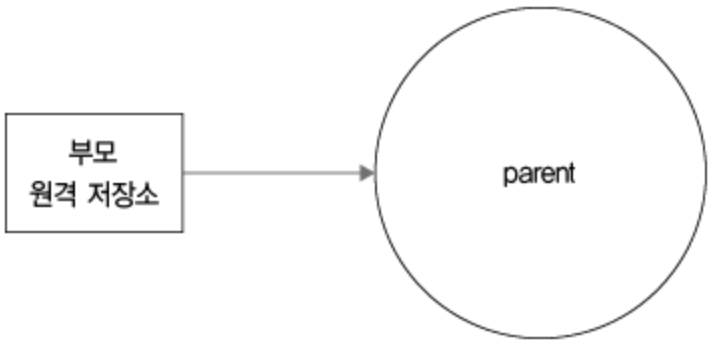

remote: Compressing objects: 100% (6/6), done.


remote: Total 8 (delta 0), reused 8 (delta 0), pack-reused 0


Unpacking objects: 100% (8/8), done.
```

gitstudy11은 부모/자식 관계인 서브모듈 형태로 되어 있습니다. 현재는 메인 저장소만 복제했으며, 그림으로 나타내면 다음과 같습니다.

▼ 그림 11-14 서브모듈 복제



11.6.2 모듈 업데이트

메인 저장소를 복제할 때는 서브모듈 정보만 같이 복제할 뿐, 실제 하위 저장소는 같이 복제하지 않습니다. 서브모듈의 하위 저장소는 직접 명령어를 실행하여 가져와야 합니다. 하위 저장소의 내용을 가져오려면 먼저 초기화와 갱신 작업을 해야 합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11/child (main)


$ cd .. 메인 저장소




infoh@DESKTOP MINGW64 /e/gitstudy11 (main)


$ git submodule init 서브모듈 초기화


Submodule 'child' (https://github.com/jinygit/gitstudy11_child.git) registered for path 'child'




infoh@DESKTOP MINGW64 /e/gitstudy11 (main)

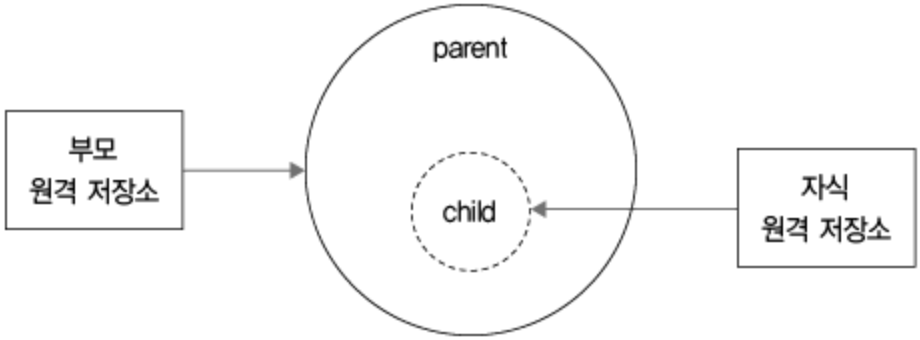

$ git submodule update 서브모듈 업데이트


Cloning into 'E:/gitstudy11/child'...


Submodule path 'child': checked out 'a7709f53d836b2abf6eb5c3d8e8064b5465458a8'
```

서브모듈 update 옵션으로 관련 하위 저장소를 복제합니다.

▼ 그림 11-15 서브모듈 초기화 및 업데이트


parent
부모
원격 저장소
child
자식
원격 저장소

11.7 부모 저장소 업데이트

메인 저장소는 서브모듈 형태로 구성되어 있습니다. 부모 저장소도 별도로 관리하므로 갱신이 필요합니다.

11.7.1 부모 업데이트

서브모듈 형태로 구성된 부모/자식 저장소도 커밋을 갱신합니다. 새로운 하위 저장소를 추가하거나 하위 저장소를 갱신할 때 메인 저장소의 설정 파일과 폴더가 변경되기 때문입니다. 따라서 자식 저장소에 코드 수정으로 커밋이 발생하면, 부모에도 자연스럽게 새로운 커밋이 발생합니다. 갱신된 메인 저장소 내용을 정기적으로 갱신하여 서브모듈 상태를 최신으로 유지해야 합니다.

11.7.2 부모 저장소로 풀

하위 저장소에 새로운 커밋이 생성되면 메인 저장소는 이를 반영합니다. 메인 저장소에는 반영한 하위 저장소의 변경된 내용 때문에 추가 커밋이 발생합니다. 자신이 메인 저장소를 복제하여 운영하고 있다면, 메인 저장소를 주기적으로 풀(pull) 작업해 주어야 합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11 (main)


$ git pull origin main
```

하지만 부모 저장소를 풀했다고 해서 부모 저장소에 종속된 모든 서브 저장소까지 자동으로 갱신되지는 않습니다. 다음과 같이 서브모듈의 업데이트는 별도로 명령어를 실행해 주어야 합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy11 (main)


$ git submodule update
```

11.8 정리

서브모듈은 규모가 큰 저장소를 작은 저장소 여러 개로 분리하여 운영할 수 있게 합니다. 또 서브모듈은 큰 프로젝트를 쪼개서 작은 프로젝트로 만들고, 저장소 크기를 줄여 가볍게 저장소를 운영할 수도 있습니다. 서브모듈 형태로 코드를 분리하면 다른 프로젝트에서도 모듈을 재사용할 수 있는 장점이 있습니다. 또 각 모듈을 원격 저장소와 연결해서 협업하여 개발을 진행할 수도 있습니다.

1 자신의 깃허브 계정 URL을 입력하세요. 책의 주소를 그대로 입력하면 권한 오류가 발생합니다.